Algoritmos

- Um algoritmo é uma sequência de passos executáveis para resolução de algum problema.

- Algoritmos fazem parte do dia a dia:
  - Instruções para uso de medicamentos
  - Manual para montar um aparelho
  - Receita de culinária

Algoritmos

- Um algoritmo é qualquer procedimento computacional que toma um valor ou conjunto de valores como entrada e produz um valor ou conjunto de valores como saída (Cormen, 2002).

- Um algoritmo corresponde a uma descrição de um padrão de comportamento, expresso em termos de um conjunto finito de ações (Dijkstra, 1971).

Estruturas de Dados

- Para resolver um problema é necessário escolher uma abstração da realidade, em geral mediante a definição de um conjunto de dados que representa a situação real.

- E então, deve ser escolhida a forma de representar esses dados.

Estruturas de Dados

- A escolha da representação dos dados é determinada, entre outras, pelas operações a serem realizadas sobre os dados.

- Exemplo: problema da ordenação (ordenar uma sequência de números em ordem crescente).
  - Entrada: Uma sequência de n números $(a_1, a_2, \ldots a_n)$
  - Saída: Uma permutação da sequência de entrada, tal que $a_1 \leq a_2 \leq \ldots \leq a_n$

Estruturas de Dados

- Uma sequência de entrada (como a lista de números a serem ordenados) é chamada de **instância** do problema.
- O melhor algoritmo para um determinado problema pode depender de vários fatores:
  - Tamanho da instância;
  - Possíveis restrições sobre os dados;
  - Espécie do dispositivo de armazenamento (memória principal ou secundária),
  - Entre outros.

Estruturas de Dados

- Um algoritmo é dito correto se, para cada entrada, ele para com a saída correta.

- Um algoritmo pode ser especificado em uma linguagem comum, como um programa de computador ou projeto de hardware.

Etapas do desenvolvimento

- Análise dos requisitos do problema
- Projeto do algoritmo
- Verificação da corretude do algoritmo
- Análise de desempenho do algoritmo
- Implementação (refinamento e codificação)
- Certificação (depuração e verificação)
- Análise de eficiência
- Documentação

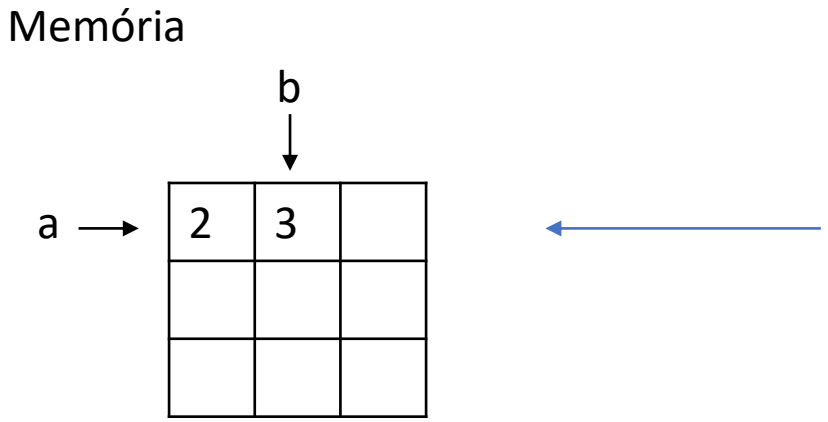
Conceitos Importantes

- Constantes:
  - Representam valores inalteráveis durante toda a execução.
- Variável:
  - Espaço da memória reservado para armazenar um certo tipo de dado e tendo um nome para referenciar o seu conteúdo.
  - Este valor pode variar durante a execução

Variáveis

- É um espaço de memória reservado para armazenar um tipo de dado.

- Vamos retomar o conceito de memória:
  - Todos os dados que nosso computador irá processar ficam na **memória principal (RAM)**:



Cada posição é um espaço na memória. Podemos dar nomes à estes espaços.

Conceitos Importantes

- Tipo de dados:
  - Define um conjunto de valores que uma variável pode armazenar
  - Define o conjunto de operações que pode ser executado com essa variável (ex.: inteiro, real, caractere).
  - Forma como a variável é armazenada e interpretada.

Conceitos Importantes

- Tipo de variáveis:
  - Informa a quantidade de memória, em bytes, que a variável ocupará, e a forma como o valor deverá ser armazenado e interpretado.
  - char: caractere de 8 bits (1 bytes).
  - int: números inteiros de 32 bits (4 bytes).
  - long: números inteiros (intervalo maior) de 64 bits (8 bytes).
  - double: ponto flutuante (de maior precisão) de 64 bits (8 bytes).

Conceitos Importantes

- Declaração de uma variável
  - **tipo_variável** nome_variável;
  - **tipo_variável** nome_variável1, nome_variável2;
  - **tipo_variável** nome_variável = valor;

- Exemplos:

```
int idade;
idade = 40;

double valor;
valor = 0.58;

char nome[] = "Alice";
```

Tipos de Estruturas

- Estrutura Condicional:
  - Comando **if**
  - Forma Geral:

    If (condição_for_verdadeira)

    comando;
  - O comando **if** executa um teste utilizando um operador relacional.
  - Condição_for_verdadeira
    - Pode ter apenas um operador relacional ou pode ter n operadores relacionais ligadas por operadores lógicos

Tipos de Estruturas

- Exemplo:

```
if (n > 0)
{
    if (a > b)
       z = a;
    else
       z = b;
}
```

Tipos de Estruturas

- Estruturas de repetição:
  - Repetir comandos um número específico de vezes ou até que uma condição conhecida ocorra.

Sintaxe do laço **for**

- Instruções: (1) inicialização, (2) teste e (3) incremento.

Inicialização | Teste | Incremento

Corpo do laço **for**. Todo o código aqui dentro irá se repetir

```
for ( i = 0;   i < 20;   i++  ) {
    ...
}
```

Laços **for** aninhados

- Podemos incluir outro laço **for** dentro do bloco de um laço **for**.

```
for (i = 0;  i < 10;  i++) {
      ...
      for (j = 0;  j < 10;  j++) {
            ...
      }
      ...
}
```

Quantas vezes uma instrução neste ponto será executada? 10 vezes.

E neste ponto? 100 vezes. Para cada vezes que o laço externo for executado, o laço interno será executado 10 vezes.

Laço **while**

- A segunda estrutura de laço de repetição é o laço **while**. O seu funcionamento é muito semelhante ao laço **for**, porém ele é mais apropriado quando ***não sabemos quantas vezes a ação será executada***. Ou seja, ele pode ser interrompido inesperadamente.

```
while (teste) {
      instrução;
      instrução;

      incremento;
      ...
}
```

Enquanto (no inglês **while**) o teste for **verdadeiro**, o programa continua a execução das instruções dentro do laço **while**. Ele irá sair do laço quando o teste for **falso**.

Observe que, para contar quantas vezes nosso laço é executado, poderemos ter um comando de incremento dentro do bloco do laço.

Laços **do-while**

- Terceira estrutura de laço de repetição. Muito similar ao laço **while** que vimos anteriormente. A diferença é que **do-while** é mais apropriado em situações em que temos que executar o corpo do laço pelo menos uma vez para depois avaliar o teste.

```
do {
      instrução;
      instrução;

      incremento;
      ...
} while (teste);
```

A palavra **do** é seguida do bloco do laço. A palavra **do** significa **faça**, ou seja, execute todo o bloco pelo menos uma vez.

Após a execução de todo o bloco, pelo menos um vez, o teste será avaliado. Enquanto o teste for **verdadeiro**, as instruções dentro do bloco será executadas mais um vez. Ele irá sair do laço quando o teste for **falso**.

O programa que conta quantidade de par

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int num;
    int par = 0;

    do {
        scanf("%d", &num);

        if ((num % 2) == 0) {
            printf("Número par detectado.\n");
            par++;
        }

    } while (num != -1);

    printf("Você digitou %d número(s) par(es).", par);

    return 0;
}
```

Vetores (Arranjos)

- Um array (vetor) é uma sequencia não ordenada de elementos do mesmo tipo.

- Arranjos podem ser:
  - Unidimensional (Vetor)
  - Bidimensional (Matriz)

- Informalmente:
  - “arranjo é uma série de variáveis **do mesmo tipo** referenciadas por um único nome”
  - cada variável (elemento do arranjo) é diferenciada por um índice

Vetores (Arranjos)

- Quando o vetor é declarado, o compilador alocará memória suficiente para conter todos os elementos do vetor.

```
int main()
{
    int vetor [5];

    int i = 0;
    for (i = 0; i < 5; i++)
        vetor[i] = (i + 1);

    for (i = 0; i < 5; i++)
        printf("%d\n", vetor[i]);

    return 0;
}
```

Arranjos Multidimensionais

- É possível definir um vetor em que cada posição temos um outro vetor (matriz). Note:

```
int matriz [3][3];
```

Arranjos Multidimensionais

```
int main()
{
    int matriz [3][3];

    int i = 0;
    int j = 0;
    int num = 0;

    for (i = 0; i < 3; i++)
        for (j = 0; j < 3; j++)
            matriz[i][j] = num++;

    for (i = 0; i < 3; i++) {
        for (j = 0; j < 3; j++)
            printf("%d ", matriz[i][j]);

        printf("\n");
    }

    return 0;
}
```

Funções e procedimentos

- A função “main” é a função principal do programa. É nesta função que todos os comandos são inicialmente executados.

- No entanto, podemos criar outras funções para manter nosso código mais organizado.

- Por exemplo, poderíamos criar uma função para a imprimir na tela a soma de dois números, e chamar esta função dentro da função principal.

Funções e procedimentos

- Observe ao lado que criamos uma função do tipo **void** que faz a soma de dois números digitados pelo usuário.

A função é do tipo **void**, significa que não retorna nenhum valor para quem a chamou. Funções do tipo **void** são chamados de procedimento. Elas executam algum tipo de processamento (como calcular uma operação entre dois números) e não retornam nada.

```
#include <stdio.h>
#include <stdlib.h>

void soma ()
{
    int a, b;

    printf("Digite o primeiro número: ");
    scanf("%d", &a);

    printf("Digite o segundo número: ");
    scanf("%d", &b);

    printf("Resultado: %d", (a + b));
}


int main()
{
    soma();
    return 0;
}
```

Funções e procedimentos

As funções e procedimentos também podem receber outras variáveis como parâmetro (argumentos). Observe que agora a leitura dos dois número é realizada na função principal e estas duas variáveis são passadas para a função como argumento.

```c
#include <stdio.h>
#include <stdlib.h>


void soma (int a, int b)
{
    printf("Resultado: %d", (a + b));
}


int main()
{
    int a, b;

    printf("Digite o primeiro número: ");
    scanf("%d", &a);

    printf("Digite o segundo número: ");
    scanf("%d", &b);

    soma(a, b);

    return 0;
}
```

Funções e procedimentos

- Podem também retornar valores para a chamada (funções). Observe que a função não é mais do tipo **void.**

```c
#include <stdio.h>
#include <stdlib.h>

int soma (int a, int b)
{
    return a + b;
}

int main()
{
    int a, b;

    printf("Digite o primeiro número: ");
    scanf("%d", &a);

    printf("Digite o segundo número: ");
    scanf("%d", &b);

    int resultado = soma(a, b);

    printf("Resultado: %d", resultado);

    return 0;
}
```

Funções e procedimentos

- Para entender melhor a questão de visibilidade na memória das variáveis, veja código ao lado onde criamos uma função que calcula o quadrado de um número.

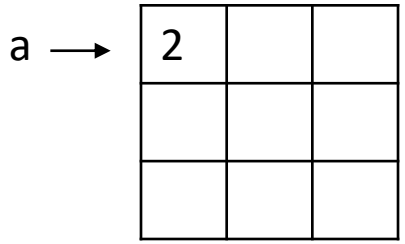
```c
#include <stdio.h>
#include <stdlib.h>

int quadrado (int a)
{
    a = a * a;
    return a;
}

int main()
{
    int a;

    printf("Digite o primeiro número: ");
    scanf("%d", &a);

    int resultado = quadrado(a);

    printf("Resultado: %d", resultado);

    return 0;
}
```

Funções e procedimentos

```c
#include <stdio.h>
#include <stdlib.h>

int quadrado (int a)
{
    a = a * a;
    return a;
}

int main()
{
    int a;

    printf("Digite o primeiro número: ");
    scanf("%d", &a);

    int resultado = quadrado(a);

    printf("Resultado: %d", resultado);

    return 0;
}
```

Suponha que a = 2

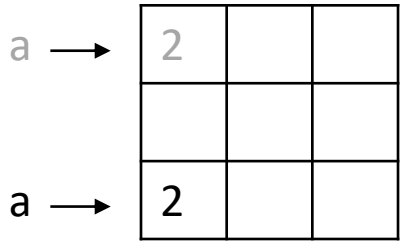
Memória



Funções e procedimentos

No momento em que a função é chamada, um novo espaço de memória é criado para a nova variável “a”, que pertence apenas à função *quadrado*. É passado como parâmetro para a função apena uma **cópia** do valor da variável.

Memória



Suponha que a = 2

```
#include <stdio.h>
#include <stdlib.h>

int quadrado (int a)
{
    a = a * a;
    return a;
}

int main()
{
    int a;

    printf("Digite o primeiro número: ");
    scanf("%d", &a);

    int resultado = quadrado(a);

    printf("Resultado: %d", resultado);

    return 0;
}
```

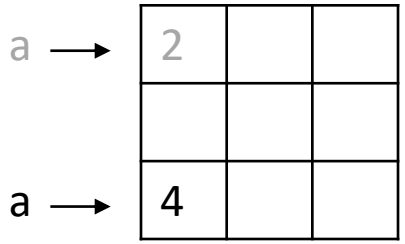
Funções e procedimentos

O valor de “a” é atualizado na memória e retornado para a função principal.

```
#include <stdio.h>
#include <stdlib.h>

int quadrado (int a)
{
    a = a * a;
    return a;
}

int main()
{
    int a;

    printf("Digite o primeiro número: ");
    scanf("%d", &a);

    int resultado = quadrado(a);

    printf("Resultado: %d", resultado);

    return 0;
}
```

Suponha que a = 2

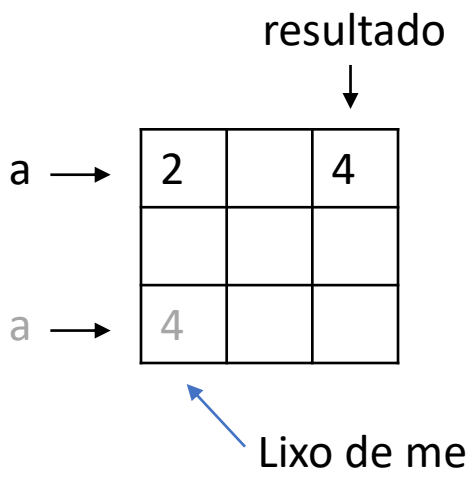
Memória



Funções e procedimentos

Observe agora que “a” pertencente a função *quadrado* fica inacessível e que a variável “resultado” criada recebeu uma cópia de seu valor. A variável “a” pertencente à função principal não sofreu qualquer alteração.

Memória



Suponha que a = 2

```c
#include <stdio.h>
#include <stdlib.h>

int quadrado (int a)
{
    a = a * a;
    return a;
}

int main()
{
    int a;

    printf("Digite o primeiro número: ");
    scanf("%d", &a);

    int resultado = quadrado(a);

    printf("Resultado: %d", resultado);

    return 0;
}
```

Exercícios

- Construa uma função **encaixa** que dados dois inteiros positivos a e b verifica se b corresponde aos últimos dígitos de a.
- Exemplo:

a	b	
567890	890	=> encaixa
1243	1243	=> encaixa
2457	245	=> não encaixa
457	2457	=> não encaixa

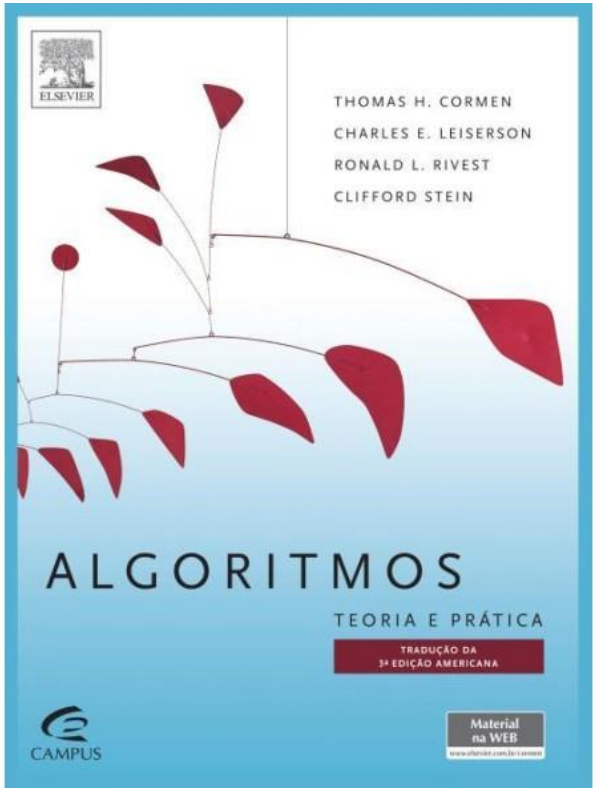
Na próxima aula

- Recursividade

Principais referências

- ZIVIANI, Nivio. Projeto de algoritmos: com implementações em Pascal e C. 3. ed. rev. e ampl. São Paulo: Cengage Learning, c2011. xx, 639 p. ISBN 9788522110506. (Disponível no Acervo). Nº de Exemplares: 13.

- http://www2.dcc.ufmg.br/livros/algoritmos/

Principais referências

- CORMEN, Thomas H. et al. Algoritmos: teoria e prática. Rio de Janeiro, RJ: Elsevier, Campus, c2012. xvi, 926 p. ISBN 9788535236996. (Disponível no Acervo). Nº de Exemplares: 4.